和豆包的对话_0319

以下5道为**超入门级string基础题**，严格按照从0到1的学习顺序设计，每题仅聚焦1-2个string核心功能，无复杂逻辑，完全适配刚接触string的初学者，所有代码均使用`using namespace std;`，符合竞赛代码习惯。

入门题1：字符串的定义与基础输入输出

考察核心功能

string的**定义、初始化、无空格输入（cin）、输出（cout）**，是学习string的第一步，对应最基础的使用方式。

题目描述

输入一个不含空格的字符串（比如你的英文名），输出固定格式的问候语：`Hello, 你输入的字符串!`

输入输出格式

- 输入：一行不含空格的字符串，长度不超过100
- 输出：一行固定格式的问候语

样例输入

```
Tom
```

样例输出

```
Hello, Tom!
```

标准题解代码

```
#include <iostream>
#include <string>  // 使用string必须包含这个头文件
using namespace std;
int main() {
// 1. 定义string类型变量，格式和普通变量完全一致：string 变量名
 string name;
// 2. 输入字符串：cin >> 变量名，自动读取不含空格的连续字符，和Python的input().split()逻辑类似
 cin >> name;
// 3. 输出字符串：cout << 内容，直接拼接输出，和Python的print用法接近
 cout << "Hello, " << name << "!" << endl;
 return 0;
}
```

核心知识点总结

1. 必须包含`<string>`头文件才能使用string类型，否则会编译报错
2. string变量无需像char数组一样提前固定长度，会自动扩容，使用更灵活
3. `cin >> s`会自动以空格、换行作为分隔符，仅读取第一个不含空格的连续字符序列，适合无空格的字符串输入

入门题2：字符串长度获取与单个字符访问

考察核心功能

string的**长度获取（size()/length()方法）、下标[]访问单个字符**，和你已经学过的数组访问逻辑完全一致，极易上手。

题目描述

输入一个不含空格的字符串，完成以下3个操作：

1. 输出字符串的总长度
2. 输出字符串的第1个字符

3. 输出字符串的最后1个字符

输入输出格式

- 输入：一行不含空格的字符串，长度不超过100
- 输出：三行，依次为总长度、第一个字符、最后一个字符

样例输入

```
Cplusplus
```

样例输出

```
9
C
s
```

标准题解代码

```
#include <iostream>
#include <string>
using namespace std;
int main() {
 string s;
 cin >> s;

// 1. 获取字符串长度：s.size() 或 s.length()，两者完全等价，竞赛中常用size()
 int len = s.size();
 cout << len << endl;

// 2. 下标访问单个字符：s[下标]，和数组完全一致，下标从0开始！
// 第一个字符的下标是0
 char first_char = s[0];
 cout << first_char << endl;
```

```
// 最后一个字符的下标是 长度-1，和数组逻辑完全一致
 char last_char = s[len - 1];
 cout << last_char << endl;

 return 0;
}
```

核心知识点总结

1. s.size()和s.length()功能完全相同，返回字符串的字符个数，初学者直接当成int用即可
2. string的下标从0开始，和数组、Python字符串完全一致，最大有效下标为s.size()-1
3. 可以通过下标直接读取单个字符，也可以后续通过下标修改字符，这是和Python字符串的核心区别（-Python字符串不可变，不能直接修改单个字符）

入门题3：字符串拼接

考察核心功能

string的**拼接运算符+、+=**，用法和Python字符串拼接几乎完全一致，无需像char数组一样手动处理-内存，非常方便。

题目描述

输入两个不含空格的字符串s1和s2，完成以下2个操作：

1. 输出s1和s2直接拼接后的新字符串（s1在前，s2在后）
2. 将s2拼接到s1的末尾，输出拼接后的s1

输入输出格式

- 输入：两行，第一行是s1，第二行是s2
- 输出：两行，第一行是s1+s2的结果，第二行是拼接后的s1

样例输入

```
Hello
World
```

样例输出

```
HelloWorld
HelloWorld
```

标准题解代码

```
#include <iostream>
#include <string>
using namespace std;
int main() {
 string s1, s2;
 cin >> s1 >> s2;

// 1. +运算符：拼接两个字符串，生成一个新的字符串，原字符串不会被修改
 string new_s = s1 + s2;
 cout << new_s << endl;

// 2. +=运算符：将右边的字符串直接拼接到左边字符串的末尾，修改原字符串
 s1 += s2;
 cout << s1 << endl;

 return 0;
}
```

核心知识点总结

1. `+`运算符：用于两个string的拼接，返回新的字符串，不会修改原来的两个字符串，和Python的`s1 + s2`完全一致
2. `+=`运算符：直接在原字符串的末尾追加内容，修改原字符串，效率比`+`更高（无需生成临时新字符串），竞赛中常用

3. 除了拼接string，还可以直接拼接字符常量，比如`s1 += "!"`、`cout << s1 + "abc" << endl;`

入门题4：字符串的修改与清空

考察核心功能

string的**单个字符修改、整体赋值、清空（clear()方法）**，充分体现C++ string的可变特性，是和-Python字符串的核心区别。

题目描述

输入一个全小写字母、不含空格的字符串，完成以下4个操作：

1. 将字符串的第一个字符改为大写，输出修改后的字符串
2. 将字符串整体赋值为`"NewString"`，输出赋值后的字符串
3. 清空字符串的所有内容，输出清空后的字符串长度
4. 判断字符串是否为空，输出`Yes`或`No`

输入输出格式

- 输入：一行全小写、不含空格的字符串，长度不超过100
- 输出：四行，依次对应上述4个操作的结果

样例输入

```
hello
```

样例输出

```
Hello
NewString
```

```
0
Yes
```

标准题解代码

```
#include <iostream>
#include <string>
#include <cctype>  // 用于toupper()函数，实现小写转大写
using namespace std;
int main() {
 string s;
 cin >> s;

// 1. 通过下标直接修改单个字符，Python中不允许直接修改字符串的单个字符
 s[0] = toupper(s[0]);  // toupper()将小写字母转为大写
 cout << s << endl;

// 2. 整体赋值：直接用=给string变量赋新值，自动覆盖原有内容
 s = "NewString";
 cout << s << endl;

// 3. 清空字符串：s.clear()，删除所有内容，长度变为0
 s.clear();
 cout << s.size() << endl;

// 4. 判断是否为空：s.empty()，为空返回true，非空返回false，比判断size()==0更高效
 if (s.empty()) {
 cout << "Yes" << endl;
} else {
 cout << "No" << endl;
}

 return 0;
}
```

核心知识点总结

1. C++ string是**可变对象**，可以通过下标直接修改单个字符，无需像Python一样重新生成新字符串，这是核心优势
2. =赋值运算符：直接给string变量赋新的字符串，原有内容会被完全覆盖，无需担心长度问题
3. s.clear()：快速清空字符串的所有内容，执行后s.size()为0，s.empty()为true
4. s.empty()：判断字符串是否为空，竞赛中优先使用，效率比s.size() == 0更高

入门题5：字符串的比较

考察核心功能

string的**比较运算符（==、!=、<、>、<=、>=）**，默认按字典序比较，用法和Python字符串比较完全一致，无需手动写循环对比字符。

题目描述

输入两个不含空格的字符串s1和s2，完成以下3个操作：

1. 判断两个字符串是否完全相等，相等输出Equal，不相等输出Not Equal
2. 若不相等，输出字典序更小的那个字符串
3. 比较两个字符串的长度，输出更长的那个字符串；长度相等则输出Same Length

输入输出格式

- 输入：两行，第一行是s1，第二行是s2
- 输出：三行，依次对应上述3个操作的结果

样例输入1

```
apple
apple
```

样例输出1

```
Equal
Same Length
```

样例输入2

```
banana
apple
```

样例输出2

```
Not Equal
apple
banana
```

标准题解代码

```
#include <iostream>
#include <string>
using namespace std;
int main() {
 string s1, s2;
 cin >> s1 >> s2;

// 1. 相等性判断：== 和 != 运算符，直接判断两个字符串的内容是否完全一致
 if (s1 == s2) {
 cout << "Equal" << endl;
} else {
 cout << "Not Equal" << endl;
// 2. 字典序比较：< 运算符，默认按ASCII码的字典序比较，和Python规则一致
 if (s1 < s2) {
 cout << s1 << endl;
} else {
```

```
 cout << s2 << endl;
}
}

// 3. 长度比较
 if (s1.size() > s2.size()) {
 cout << s1 << endl;
} else if (s1.size() < s2.size()) {
 cout << s2 << endl;
} else {
 cout << "Same Length" << endl;
}

 return 0;
}
```

核心知识点总结

1. string的比较运算符直接对比**字符串的内容**，而非地址，和char数组的指针比较完全不同，初学者无需担心踩坑
2. 字典序比较规则：从左到右依次对比每个字符的ASCII码值，第一个不同的字符决定了两个字符串的大小，和Python、字典的排序规则完全一致
3. 注意区分：`s1 < s2`是字典序比较，`s1.size() < s2.size()`是长度比较，两者没有必然联系

完成以上5道题，你就掌握了string最核心的基础操作，接下来就可以按顺序完成之前的10道进阶题，逐步适配算法竞赛的常用场景。

(豆包AI生成)